

1. Теория вероятностей

Базовые определения

- Вероятность $P(A)$ — число от 0 до 1, $P(\Omega)=1$, $P(\emptyset)=0$
- Случайное событие $A \subseteq \Omega$ (пространство элементарных исходов)
- Независимые события: $P(A \cap B) = P(A) \cdot P(B)$

Ключевые формулы

Формула	Описание
Сложение (несовм.)	$P(A \cup B) = P(A) + P(B)$
Сложение (совм.)	$P(A \cup B) = P(A) + P(B) - P(A \cap B)$
Условная вер-ть	$P(A B) = P(A \cap B) / P(B)$
Независимость	$P(A \cap B) = P(A) \cdot P(B)$
Полная вероятность	$P(B) = \sum_i P(B A_i) \cdot P(A_i)$
Формула Байеса	$P(A_i B) = P(B A_i) \cdot P(A_i) / P(B)$
Включений-исключений	$P(A \cup B \cup C) = \sum P(A_i) - \sum P(A_i \cap A_j) + P(A \cap B \cap C)$

Характеристики случайной величины

Формула	Описание
Математич. ожидание	$E[X] = \sum x_i \cdot p(x_i)$ (дискретная)
Дисперсия	$\text{Var}(X) = E[X^2] - (E[X])^2$
Среднеквадрат. откл.	$\sigma = \sqrt{\text{Var}(X)}$
Ковариация	$\text{Cov}(X, Y) = E[XY] - E[X] \cdot E[Y]$
Линейность E	$E[aX + b] = a \cdot E[X] + b$
Var линейная	$\text{Var}(aX + b) = a^2 \cdot \text{Var}(X)$

Основные распределения

Формула	Описание
Бернулли	$P(X=1)=p, P(X=0)=1-p; E=p, \text{Var}=p(1-p)$
Биномиальное	$P(X=k)=C(n,k)p^k(1-p)^{n-k}; E=np$
Нормальное $N(\mu, \sigma^2)$	$f(x)=(1/\sigma\sqrt{2\pi}) \cdot \exp(-(x-\mu)^2/2\sigma^2)$
Пуассона	$P(X=k)=\lambda^k e^{-\lambda}/k!; E=\lambda, \text{Var}=\lambda$

2. Линейная алгебра

Векторы

Операция	Формула / Код
Норма (длина)	$\ v\ = \sqrt{v_1^2 + v_2^2 + \dots + v_n^2}$
Скалярное произведение	$a \cdot b = \sum a_i b_i = \ a\ \cdot \ b\ \cdot \cos(\theta)$
Единичный вектор	$\hat{v} = v / \ v\ $
Косинусное сходство	$\cos(\theta) = (a \cdot b) / (\ a\ \cdot \ b\)$

Матрицы

Операция	Формула / Код
Транспонирование	$(AB)^T = B^T A^T$; $(A^T)^T = A$
Обратная матрица	$A \cdot A^{-1} = I$; $(AB)^{-1} = B^{-1} A^{-1}$
Определитель 2x2	$\det([[a,b],[c,d]]) = ad - bc$
Rank (ранг)	Кол-во линейно независимых строк/столбцов
Собственные значения	$A \cdot v = \lambda \cdot v$; $\det(A - \lambda I) = 0$
Trace (след)	$\text{tr}(A) = \sum a_{ii} = \sum \lambda_i$

Разложения матриц

Операция	Формула / Код
SVD	$A = U \cdot \Sigma \cdot V^T$ (U,V — ортогональные; Σ — диагональная)
Eigen decomposition	$A = Q \cdot \Lambda \cdot Q^{-1}$
PCA шаги	1) центрировать 2) ковар. матрица 3) eigen 4) проекция

NumPy — линейная алгебра

```
import numpy as np
A @ B                # матричное умножение
np.dot(a, b)         # скалярное/матр. произведение
np.linalg.inv(A)      # обратная матрица
np.linalg.det(A)      # определитель
np.linalg.eig(A)      # собственные значения и векторы
np.linalg.svd(A)      # SVD разложение
np.linalg.norm(v)     # норма вектора
A.T                  # транспонирование
```

3. Статистика

Меры центральной тенденции

Метрика	Формула	Когда использовать
Mean (среднее)	$\mu = \sum x_i / n$	Числовые данные без выбросов
Median (медиана)	центральный элемент	При наличии выбросов
Mode (мода)	наибольшая частота	Категориальные данные
Геом. среднее	$\sqrt[n]{x_1 \cdot x_2 \cdot \dots \cdot x_n}$	Темпы роста, проценты

Меры разброса

Дисперсия	$\sigma^2 = \sum (x_i - \mu)^2 / n$ (ген.) ; $s^2 = \sum (x_i - \bar{x})^2 / (n-1)$ (выб.)
Стандартное откл.	$\sigma = \sqrt{\text{дисперсия}}$ — в тех же единицах, что x
IQR	$Q3 - Q1$ (межквартильный размах)
Выброс (правило)	$< Q1 - 1.5 \cdot IQR$ или $> Q3 + 1.5 \cdot IQR$
Коэф. вариации	$CV = \sigma / \mu \cdot 100\%$

Типы переменных

Числовые непрерывные	рост, вес, температура — дробные значения
Числовые дискретные	кол-во покупок, оценки — целые числа
Категориальные номинальные	цвет, пол, город — нет порядка
Категориальные порядковые	XS/S/M/L, оценка 1-5 — есть порядок

Корреляция

Пирсон r	$\sum [(x_i - \bar{x})(y_i - \bar{y})] / (n \cdot \sigma_x \cdot \sigma_y)$ — линейная связь
Диапазон	-1 (обратная) ... 0 (нет) ... +1 (прямая)
Сильная связь	$ r > 0.7$
Слабая связь	$ r < 0.3$
Ограничение	Корреляция $\neq$ Причинность!

Проверка гипотез — ключевые понятия

H_0 (нулевая)	Нет эффекта / нет разницы
H_1 (альтернативная)	Есть эффект / есть разница
p-value	$P(\text{данные} H_0)$ — вероятность при условии H_0
Уровень значимости α	Обычно 0.05 — порог отклонения H_0
Ошибка I рода	Отклонить H_0 когда она верна (ложная тревога)
Ошибка II рода	Принять H_0 когда она неверна (пропуск эффекта)
Мощность критерия	$1 - P(\text{ошибки II рода})$

Пропущенные значения

Обнаружение	<code>df.isnull().sum()</code> / <code>df.info()</code>
-------------	---

Удаление строк	<code>df.dropna()</code>
Заполнение средним	<code>df.fillna(df.mean())</code>
Заполнение медианой	<code>df.fillna(df.median())</code> — при выбросах
Заполнение модой	<code>df['col'].fillna(df['col'].mode()[0])</code> — категории
Интерполяция	<code>df.interpolate()</code> — временные ряды

4. Визуализация

Выбор типа графика

График	Когда использовать	Пример данных
Гистограмма	Распределение 1 числовой переменной	<code>plt.hist(df['age'])</code>
Boxplot (ящик)	Медиана, квартили, выбросы	<code>sns.boxplot(y=df['salary'])</code>
Violin plot	Boxplot + плотность распределения	<code>sns.violinplot(...)</code>
Scatter plot	Связь 2 числовых переменных	<code>plt.scatter(x, y)</code>
Bubble chart	Scatter + 3-я переменная = размер	<code>plt.scatter(x, y, s=size)</code>
Line plot	Динамика / временной ряд	<code>plt.plot(dates, values)</code>
Bar chart	Сравнение категорий	<code>sns.barplot(x='cat', y='val')</code>
Heatmap	Матрица корреляций / сводные таблицы	<code>sns.heatmap(df.corr())</code>
Pie chart	Доли целого (не больше 5-6 категорий)	<code>plt.pie(values)</code>
KDE plot	Плотность вероятности (гладкая)	<code>sns.kdeplot(df['x'])</code>
Pairplot	Все попарные зависимости сразу	<code>sns.pairplot(df)</code>

Чтение boxplot

- Центральная линия = медиана (Q2)
- Нижняя граница ящика = Q1 (25-й перцентиль)
- Верхняя граница ящика = Q3 (75-й перцентиль)
- Усы = $Q1 - 1.5 \cdot IQR$ и $Q3 + 1.5 \cdot IQR$
- Точки за усами = выбросы

Чтение графика плотности (KDE / CDF)

- PDF (плотность): площадь под кривой = 1; пик = мода
- CDF (функция распределения): значение в точке $x = P(X \leq x)$
- CDF монотонно растёт от 0 до 1

Основной код

```
import matplotlib.pyplot as plt
import seaborn as sns
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
sns.pairplot(df, hue='target')
plt.figure(figsize=(10,6)); plt.show()
```

5. Работа с данными (Pandas)

Первичный осмотр

```
df.head(5)          # первые 5 строк
df.tail(5)          # последние 5 строк
df.shape            # (строки, столбцы)
df.info()           # типы, кол-во non-null
df.describe()       # статистики числовых столбцов
df.isnull().sum()   # кол-во NaN по столбцам
df['col'].value_counts() # частоты значений
```

Фильтрация и выборка

```
df[df['age'] > 30]          # условие
df[(df['a'] > 0) & (df['b'] == 'Y')] # И
df[df['city'].isin(['Москва', 'СПб'])] # в списке
df.loc[0:5, ['col1', 'col2']]      # по метке
df.iloc[0:5, 0:3]                  # по индексу
```

Создание и изменение столбцов

```
df['new_col'] = df['a'] / df['b']
df['col'] = df['col'].apply(lambda x: x*2)
df['col'] = df['col'].astype(int)
df.rename(columns={'old': 'new'}, inplace=True)
df.drop(columns=['col1', 'col2'], inplace=True)
```

Агрегация и группировка

```
df.groupby('city')['salary'].mean()
df.groupby('city').agg({'sal': 'mean', 'age': 'max'})
df.pivot_table(values='sal', index='dept', aggfunc='mean')
df.sort_values('salary', ascending=False)
```

Работа с пропусками

```
df.dropna()          # удалить строки с NaN
df.fillna(0)         # заполнить нулями
df['col'].fillna(df['col'].mean()) # заполнить средним
df['col'].fillna(method='ffill')  # заполнить предыдущим
```

Корреляция

```
df.corr()            # матрица корреляций Пирсона
df.corr(method='spearman') # ранговая (устойчива к выбросам)
df['a'].corr(df['b'])   # корреляция двух столбцов
```

6. Машинное обучение — Алгоритмы

Линейная регрессия

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n = Xw$$

$$\text{MSE Loss} = (1/n) \cdot \sum (y_i - \hat{y}_i)^2$$

$$\text{Градиентный спуск: } w := w - \alpha \cdot \nabla L(w)$$

- Ridge (L2): добавляет $\lambda \cdot \|w\|^2$ к loss → уменьшает веса
- Lasso (L1): добавляет $\lambda \cdot \|w\|_1$ к loss → обнуляет веса (отбор признаков)

Логистическая регрессия (классификация)

$$\sigma(z) = 1 / (1 + e^{-z}) \quad - \text{СИГМОИД}$$

$$P(y=1|x) = \sigma(w \cdot x + b)$$

$$\text{Log Loss} = -(1/n) \cdot \sum [y_i \cdot \log(\hat{p}_i) + (1-y_i) \cdot \log(1-\hat{p}_i)]$$

KNN (К ближайших соседей)

- Расстояние Евклида: $d = \sqrt{\sum (x_i - y_i)^2}$
- Малое k → сложная граница → overfitting
- Большое k → гладкая граница → underfitting
- Нужна нормализация признаков (StandardScaler)

Решающее дерево

$$\text{Gini} = 1 - \sum p_i^2$$

$$\text{Entropy} = -\sum p_i \cdot \log_2(p_i)$$

$$\text{Information Gain} = H(\text{parent}) - \sum w_i \cdot H(\text{child}_i)$$

- Глубокое дерево → overfitting; ограничивают max_depth
- Интерпретируемо, не требует нормализации

Ансамблевые методы

Bagging	Обучение на случайных подвыборках (с возвратом), усреднение
Random Forest	Bagging + случайный выбор признаков на каждом сплите
Boosting	Каждый следующий алгоритм обучается на ошибках предыдущего
Gradient Boosting	Градиентный спуск в пространстве функций; строит по остаткам
Bias-Variance	Bagging ↓ Variance; Boosting ↓ Bias; Stacking — оба

Разложение ошибки

$$\text{Error} = \text{Bias}^2 + \text{Variance} + \text{Noise}$$

- Bias — систематическая ошибка (модель слишком проста)
- Variance — чувствительность к шуму обучающей выборки
- Noise — неустраняемая ошибка в данных

7. KNN — алгоритм по шагам

Идея

- Для новой точки найти k ближайших соседей из обучающей выборки
- Классификация: класс который чаще всего встречается среди k соседей (голосование)
- Регрессия: среднее значение среди k соседей
- Важно: нужна нормализация признаков перед применением!

Евклидово расстояние — формула

$$d(A, B) = \sqrt{(x1_A - x1_B)^2 + (x2_A - x2_B)^2}$$

Пример: A=(-1,1), D=(0,0)

$$d(A, D) = \sqrt{((-1-0)^2 + (1-0)^2)} = \sqrt{1+1} = \sqrt{2} \approx 1.41$$

Leave-One-Out кросс-валидация — алгоритм

- Шаг 1: Убрать одну точку из датасета
- Шаг 2: Обучить KNN на оставшихся точках
- Шаг 3: Предсказать класс убранный точки
- Шаг 4: Сравнить предсказание с правильным ответом
- Шаг 5: Повторить для каждой точки
- Шаг 6: Accuracy = кол-во верных предсказаний / всего точек

Пример: 4 точки, ищем оптимальное k

Точки	A=(-1,1) класс 1 B=(1,-1) класс 1 C=(1,1) класс 1 D=(0,0) класс -1
Все расстояния	A↔D=1.41 B↔D=1.41 C↔D=1.41 A↔C=2 A↔B=2.83 B↔C=2

k=1 — результаты

Убираем A	ближайшая → D(1.41), класс -1 правда=1 ❌
Убираем B	ближайшая → D(1.41), класс -1 правда=1 ❌
Убираем C	ближайшая → D(1.41), класс -1 правда=1 ❌
Убираем D	ближайшие → A,B,C все 1.41, класс 1 правда=-1 ❌
Accuracy k=1	0/4 = 0.0

k=2 — результаты

Убираем A	2 ближайших: D(-1), C(1) → ничья → 1 правда=1 ✅
Убираем B	2 ближайших: D(-1), C(1) → ничья → 1 правда=1 ✅
Убираем C	2 ближайших: D(-1), A(1) → ничья → 1 правда=1 ✅
Убираем D	2 ближайших: A(1), B(1) → голос. → 1 правда=-1 ❌
Accuracy k=2	3/4 = 0.75

k=3 — результаты

Убираем A	3 ближайших: D(-1), C(1), B(1) → большинство 1 правда=1 ✅
-----------	--

Убираем В	3 ближайших: D(-1), C(1), A(1) → большинство 1 правда=1 
Убираем С	3 ближайших: D(-1), A(1), B(1) → большинство 1 правда=1 
Убираем D	3 ближайших: A(1), B(1), C(1) → все 1 правда=-1 
Ассурасу k=3	3/4 = 0.75

Итог и вывод

k=1	Accuracy = 0.0
k=2	Accuracy = 0.75
k=3	Accuracy = 0.75
Оптимальное k	k=2 (при равных ассурасу берём меньшее k — проще модель)

Как выбрать k на экзамене

- Маленькое k (1-3) → сложная граница → может переобучиться
- Большое k → гладкая граница → может недообучиться
- При ничье в голосовании → берём класс 1 (или меньший по номеру)
- При равных ассурасу → выбираем меньшее k

7. Метрики качества

Матрица ошибок (классификация)

TP	True Positive – правильно предсказан класс 1
TN	True Negative – правильно предсказан класс 0
FP	False Positive – предсказали 1, на самом деле 0 (ошибка I рода)
FN	False Negative – предсказали 0, на самом деле 1 (ошибка II рода)

Метрики классификации

Accuracy	$(TP+TN) / (TP+TN+FP+FN)$
Precision	$TP / (TP+FP)$ – из предсказанных 1, сколько верны
Recall (TPR)	$TP / (TP+FN)$ – из реальных 1, сколько нашли
F1-score	$2 \cdot P \cdot R / (P+R)$ – гармоническое среднее P и R
ROC-AUC	Площадь под ROC-кривой (TPR vs FPR); 0.5=случайно, 1=идеал
Когда Precision	Дорого ложная тревога: спам, мошенничество
Когда Recall	Дорого пропуск: болезни, безопасность

Метрики регрессии

MAE	$(1/n) \cdot \sum y_i - \hat{y}_i $ – в единицах y , устойчив к выбросам
MSE	$(1/n) \cdot \sum (y_i - \hat{y}_i)^2$ – штрафует большие ошибки
RMSE	$\sqrt{MSE}$ – в единицах y
R ² (R-squared)	$1 - SS_{res}/SS_{tot}$; 1=идеал, 0=константа, <0 хуже конст.
MAPE	$(1/n) \cdot \sum y_i - \hat{y}_i / y_i \cdot 100\%$ – относительная ошибка

Метрики кластеризации

Silhouette score	$s = (b-a)/\max(a,b)$; диапазон [-1,1]; ближе к 1 = лучше
Inertia (SSE)	$\sum$ расстояний от точек до центроидов кластеров
Adjusted Rand	Сравнение с истинными метками; 1=идеал, 0=случайно

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import confusion_matrix, roc_auc_score
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.metrics import silhouette_score
```

8. Оценка и улучшение модели

Overfitting / Underfitting

Underfitting	Train↓ Val↓ — модель слишком проста; увеличить сложность
Overfitting	Train↑ Val↓ — модель запомнила шум; регуляризация, больше данных
Хорошая модель	Train ≈ Val (оба высокие)

Кросс-валидация (K-Fold)

- Данные делятся на K частей (folds)
- Модель обучается на K-1 fold, оценивается на 1 fold
- Процесс повторяется K раз, результат усредняется
- Стандарт: K=5 или K=10

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(model, X, y, cv=5, scoring='accuracy')
print(scores.mean(), scores.std())
```

GridSearchCV — подбор гиперпараметров

```
from sklearn.model_selection import GridSearchCV
params = {'max_depth': [3,5,7], 'n_estimators': [100,200]}
grid = GridSearchCV(model, params, cv=5, scoring='f1')
grid.fit(X_train, y_train)
print(grid.best_params_, grid.best_score_)
```

Sklearn — стандартный пайплайн

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test) # только transform!
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Кодирование категориальных переменных

LabelEncoder	0,1,2,... — для ординальных или древесных моделей
OneHotEncoder	N-1 бинарных столбцов — для номинальных переменных
get_dummies	pd.get_dummies(df, drop_first=True) — просто и быстро

```
from sklearn.preprocessing import LabelEncoder, OneHotEncoder
le = LabelEncoder(); y = le.fit_transform(y)
pd.get_dummies(df[['city','gender']], drop_first=True)
```

9. Кластеризация

K-Means

- Задать k кластеров, случайно выбрать центроиды
- Присвоить каждую точку ближайшему центроиду
- Пересчитать центроиды как среднее кластера
- Повторять до сходимости

Цель: минимизировать $\sum \sum ||x_i - \mu_k||^2$

- Выбор k: Elbow method (inertia vs k) или Silhouette score

```
from sklearn.cluster import KMeans
km = KMeans(n_clusters=3, random_state=42)
labels = km.fit_predict(X)
km.inertia_ # сумма квадратов расстояний
```

DBSCAN

eps	Радиус окрестности — если маленький, много кластеров
min_samples	Мин. точек для плотного региона — если большой, меньше кластеров
Шум	Точки, не входящие ни в один кластер (метка = -1)
Преимущество	Находит кластеры произвольной формы; устойчив к выбросам

```
from sklearn.cluster import DBSCAN
db = DBSCAN(eps=0.5, min_samples=5)
labels = db.fit_predict(X)
# labels == -1 это шум
```

Метрики кластеризации

Silhouette: $s(i) = (b(i) - a(i)) / \max(a(i), b(i))$

- $a(i)$ — среднее расстояние до точек своего кластера
- $b(i)$ — среднее расстояние до ближайшего чужого кластера
- $s \approx 1$: точка хорошо кластеризована; $s \approx -1$: попала не в тот кластер

10. Тексты и изображения

Обработка текста

Токенизация	Разбить текст на токены (слова/подслова/символы)
Нормализация	Нижний регистр, удаление пунктуации и спецсимволов
Stop words	Удаление частых незначимых слов (и, или, но, the...)
Стемминг	Обрезка окончаний: 'бежит' → 'беж' (быстро, неточно)
Лемматизация	Приведение к лемме: 'бежит' → 'бежать' (точнее)

Векторизация текста

Bag of Words	Матрица частот слов; не учитывает порядок
TF-IDF	$TF \cdot \log(N/df)$; TF=частота в документе, df=в скольких документах
TF формула	$tf(t,d) = \text{count}(t,d) / \text{len}(d)$
IDF формула	$idf(t) = \log(N / df(t))$; N — всего документов

```
from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer
vec = TfidfVectorizer(max_features=5000, stop_words='english')
X = vec.fit_transform(corpus)
```

Изображения — основы

Пиксель	Число от 0 (чёрный) до 255 (белый) для grayscale
RGB изображение	3 канала (R, G, B), форма: (H, W, 3)
Grayscale	1 канал, форма: (H, W)
Нормализация	$\text{pixel} / 255.0 \rightarrow \text{значения } [0,1]$
Стандартизация	$(\text{pixel} - \text{mean}) / \text{std} \rightarrow \text{нулевое среднее}$
Flatten	Развернуть в 1D вектор для линейных моделей

```
import numpy as np
img = img / 255.0 # нормализация
gray = img.mean(axis=2) # RGB → grayscale (приблизительно)
flat = img.reshape(img.shape[0], -1) # flatten
```

11. Нейронные сети

Типы слоёв

Dense (Fully Connected)	Каждый нейрон связан со всеми нейронами следующего слоя
Conv2D (Свёрточный)	Применяет фильтры к локальным регионам изображения
MaxPooling	Уменьшает размерность; берёт максимум в окне
Dropout	Случайно обнуляет нейроны при обучении → борьба с overfitting
BatchNormalization	Нормализует активации → ускоряет обучение
Flatten	Разворачивает многомерный тензор в вектор
Embedding	Преобразует целые числа (токены) в векторы

Функции активации

ReLU	$\max(0, x)$ — стандарт для скрытых слоёв; быстрая, простая
Sigmoid	$1/(1+e^{-x}) \rightarrow (0,1)$ — для бинарной классификации (выход)
Softmax	$e^{x^i}/\sum e^{x^j} \rightarrow$ вероятности классов — для многоклассовой (выход)
Tanh	$(e^{2x}-1)/(e^{2x}+1) \rightarrow (-1,1)$ — центрирована, для RNN
Leaky ReLU	x если $x>0$; αx если $x<0$ — не 'умирает' как ReLU

Функции потерь

Binary Cross-Entropy	Бинарная классификация (sigmoid на выходе)
Categorical Cross-Entropy	Многоклассовая (softmax на выходе)
MSE	Регрессия — штрафует большие ошибки
MAE	Регрессия — устойчива к выбросам

Ключевые понятия

Epoch	Полный проход по всем обучающим данным
Batch size	Кол-во примеров за один шаг градиентного спуска
Learning rate	Размер шага градиентного спуска (обычно 0.001–0.01)
Backpropagation	Вычисление градиентов через цепное правило

12. Важные факты с практики

Violin plot — скрипичный график

Что это	Жёлтая фигура = KDE плотность; внутри встроен boxplot
Ширина фигуры	Чем шире — тем больше данных в этом месте
Мода	Самое широкое место скрипки = мода
2 моды	Скрипка имеет ДВА горба/пика — не две палочки в ящике!
Медиана	Вертикальная линия ВНУТРИ синего прямоугольника
Выбросы	Скрипка тянется за усы боксплота
IQR	Ширина прямоугольника ($Q3-Q1$), НЕ вся шкала

Нормальное распределение — правило sigma

$\mu \pm 1\sigma$	$34.1\% + 34.1\% = 68.2\%$ данных
$\mu \pm 2\sigma$	$13.6\% + 68.2\% + 13.6\% = 95.4\%$ данных
$\mu \pm 3\sigma$	$2.1\% + 95.4\% + 2.1\% = 99.7\%$ данных
Правее $+3\sigma$	0.1% данных
От μ до $+2\sigma$	$34.1\% + 13.6\% = 47.7\%$
Перевод	Сначала найди: $(x - \mu) / \sigma = \text{кол-во сигм}$

Геометрическое распределение

Когда	Кол-во попыток до первого успеха
$E[X]$	$1 / p$
Пример	$p=0.05 \rightarrow E[X] = 1/0.05 = 20$ попыток в среднем

ROC-AUC — ключевые факты

Что измеряет	Вер-ть что объект класса 1 получит score ВЫШЕ чем класс 0
0.5	Случайный классификатор
1.0	Идеальная модель
Зависит от	Только от ПОРЯДКА предсказаний, не от значений
Монотонное преобр.	x^2 на $[0,1]$ — порядок не меняется $\rightarrow$ AUC не меняется
Формула	$AUC = \text{пары где } score(1) > score(0) / (n1 * n0)$
Сортировка вверх	Объекты класса 0 ЛЕВее объекта класса 1 $\rightarrow$ засчитываются

Корреляция Пирсона — ловушки

Измеряет	Только ЛИНЕЙНУЮ связь
$r = 0$	НЕ значит независимость! Может быть нелинейная связь
Окружность	$x1^2 + x2^2 = 1 \rightarrow r \approx 0$, но $x1$ и $x2$ зависимы нелинейно
Главное	Корреляция НЕ равно причинность!

Окружность — ошибочные утверждения (частая ловушка)

ОШИБКА: линейно зависимы	Связь есть, но она нелинейная (квадратичная)
--------------------------	--

ОШИБКА: независимы	Зная x_1 , знаем $ x_2 = \sqrt{1-x_1^2}$ — зависимы!
ОШИБКА: можно удалить признак	Слово 'любого' алгоритма делает неверным — нелинейные алгоритмы используют оба
ВЕРНО: $r \approx 0$	Симметрия окружности гасит линейную связь
ВЕРНО: зависимы нелинейно	$x_1^2 + x_2^2 = 1$ — нелинейная зависимость

Precision vs Specificity — частая путаница

Precision $TP/(TP+FP)$	Из предсказанных 1 — сколько верных; штрафует FP со стороны класса 1
Specificity $TN/(FP+TN)$	Из реальных 0 — сколько верно распознано; штрафует FP со стороны класса 0
Пример	Чужой голос (класс 0) → нужна Specificity, не Precision
Правило	Если задача про класс 0 → Specificity; про класс 1 → Precision

Логистическая регрессия без регуляризации

1 категор. признак	Предсказание = просто пропорция из данных
Пример	35 из 52 пешком приняты → $P = 35/52 \approx 0.67$
С регуляризацией	Результат тянется к среднему по всей выборке
w_0 (bias/intercept)	НЕ увеличивает переобучение — убирать не надо

Градиентный спуск — важные факты

Цель	Найти МИНИМУМ функции потерь (не максимум!)
Batch GD	Градиент по ВСЕМ объектам за шаг
SGD	Градиент по ОДНОМУ случайному объекту за шаг
Большой шаг	Алгоритм может РАЗОЙТИСЬ
Малый шаг	Медленная сходимость

Случайный лес — важные факты

Снижает	ДИСПЕРСИЮ (variance), не смещение!
Bias	Примерно такой же как у одного дерева
Bootstrap	Выборка С ВОЗВРАЩЕНИЕМ (повторения есть!) $\approx 63\%$ уникальных
Признаки	На каждом сплите берётся случайное подмножество
Кол-во деревьев	Не переобучается с ростом (в отличие от бустинга)

Регуляризация — шпаргалка

L1 (Lasso)	$ w_1 + w_2 $ — обнуляет веса → отбор признаков
L2 (Ridge)	$w_1^2 + w_2^2$ — уменьшает веса → не обнуляет
w_0 (bias)	НЕ увеличивает переобучение — убирать не нужно
Полином. признаки	УВЕЛИЧИВАЮТ сложность → усиливают переобучение
L0	Не дифференцируема → на практике не используется

Sklearn — стандартный код для экзамена

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score, GridSearchCV
from sklearn.metrics import roc_auc_score
```

Кросс-валидация с метрикой:

```
dt = DecisionTreeClassifier(max_depth=5, criterion='entropy')
scores = cross_val_score(dt, X_train, y_train, cv=3, scoring='roc_auc')
print(round(scores.mean(), 1))
```

GridSearch для подбора глубины:

```
param_grid = {'max_depth': list(range(2, 21))}
dt = DecisionTreeClassifier() # random_state=None по умолчанию!
grid = GridSearchCV(dt, param_grid, cv=3, scoring='roc_auc')
grid.fit(X_train, y_train)
print(grid.best_params_['max_depth'])
```

ВАЖНО: не указывай лишние параметры если в задании сказано 'дефолтные'
random_state=42 изменит результат если в условии random_state=None!

One-Hot Encoding — типичные ошибки

drop_first=True	ОБЯЗАТЕЛЬНО если не хотим мультиколлинеарности
Кол-во новых столбцов	Для каждого столбца: уникальных_значений - 1
KeyError после get_dummies	Столбцы уже закодированы! Перечитай данные заново
Числовые столбцы	НЕ кодируй get_dummies — только категориальные (dtype==object)

```
pd.get_dummies(df, columns=['type', 'meal'], drop_first=True, dtype=int)
```

Типичные ошибки в pandas

fillna без присваивания	df['col'].fillna(0) — НЕ меняет df! Нужно df['col'] = df['col'].fillna(0)
count() vs sum()	.count() = кол-во непустых; .sum() = сумма (для True/False)
score-1 дважды	Проверяй опечатки: score-2 и score-2 вместо score-3
ROC-AUC = 1.0	Скорее всего target утёк в признаки! Проверь X_train.columns
score в X_train	Pass создан из score → score надо убрать из X

Data Leakage — золотое правило

Правило	Все параметры считаются ТОЛЬКО из train
fillna числовой	mean = df_train.mean() → применять к train И test
Scaler	fit(X_train) → transform(X_train) и transform(X_test)
Почему	Test = будущие данные; их статистики нам неизвестны
Последствие	Утечка → модель кажется лучше чем есть на самом деле

Код — заполнение пропусков правильно

```
for i in df_train.columns:
    if df_train[i].dtype == 'object': # категориальный
```

```

df_train[i] = df_train[i].fillna('Unknown')
df_test[i] = df_test[i].fillna('Unknown')
else:
    # числовой
    mean_val = df_train[i].mean() # ТОЛЬКО из train!
    df_train[i] = df_train[i].fillna(mean_val)
    df_test[i] = df_test[i].fillna(mean_val)

```

Ширина полосы между гиперплоскостями

```

ширина = |c1 - c2| / ||w||

```

Пример: $w=(2,3)$, $c1=8$, $c2=6 \rightarrow \text{ширина} = 2/\sqrt{13} \approx 0.55$

Формула Байеса — шаги решения

- Шаг 1: $P(\text{класс})$ для каждого класса
- Шаг 2: $P(\text{событие}|\text{класс})$ для каждого класса
- Шаг 3: $P(\text{событие}) = \text{сумма } P(\text{событие}|\text{класс}_i) \cdot P(\text{класс}_i)$
- Шаг 4: $P(\text{класс}|\text{событие}) = P(\text{событие}|\text{класс}) \cdot P(\text{класс}) / P(\text{событие})$

Пример: $P(A|\text{авария}) = 0.01 \cdot 0.30 / 0.038 \approx 0.08$